

Questões de Prova - Seminário sobre FlashAttention

QUESTÃO 1

ENUNCIADO:

Por que a implementação padrão da atenção é considerada "IO-unaware" (ignorante em I/O) e qual o principal problema de desempenho que isso causa?

RESPOSTA:

A atenção padrão é considerada "IO-unaware" (ignorante em I/O) porque seu algoritmo não é projetado levando em conta a hierarquia de memória das GPUs. Ele trata a memória da GPU como um bloco único, ignorando a diferença significativa de desempenho entre a memória principal e a memória no chip. A memória principal da GPU é grande (podendo chegar a dezenas de gigabytes), mas comparativamente lenta, especialmente em termos de latência (centenas de ciclos). A memória no chip é minúscula (centenas de kilobytes por unidade de computação), mas ultra-rápida, com latência baixíssima (dezenas de ciclos) e enorme largura de banda.

O problema é que a atenção padrão materializa as matrizes intermediárias S (de escores de atenção) e P (de escores normalizados), ambas de tamanho $N \times N$, na memória. Como N (comprimento da sequência) cresce, essas matrizes se tornam gigantescas e não cabem na memória rápida, forçando o algoritmo a escrevê-las e lê-las da memória lenta múltiplas vezes. O fluxo de I/O é o seguinte: (1) ler os inputs Q (query) e K (key) da memória lenta; (2) escrever a matriz S na memória lenta; (3) ler a matriz S da memória lenta para aplicação do softmax; (4) escrever a matriz P , resultante da aplicação do softmax, na memória lenta; (5) ler a matriz P e o input V (value) da memória lenta; (6) escrever o resultado da atenção na memória lenta. Esse excesso de leituras e escritas na memória lenta é o verdadeiro gargalo. A GPU, com suas potentes unidades de computação, passa a maior parte do tempo ociosa, esperando dados serem lidos da ou escritos na memória lenta. Esse problema é conhecido como "memory-bound" (limitado pela memória), onde o tempo de execução é ditado pela largura de banda da memória, e não pela velocidade de cálculo da GPU.

QUESTÃO 2

ENUNCIADO:

O FlashAttention precisa processar os dados em blocos (tiling). Como ele consegue calcular o softmax, que precisa carregar toda uma linha de dados da matriz S (de escores de atenção, com tamanho $N \times N$) para calcular o denominador naquela mesma linha?

RESPOSTA:

O desafio do softmax padrão é que ele requer carregar e conhecer todos os elementos de uma linha da matriz S para calcular o denominador naquela mesma linha. Além disso, para estabilidade numérica, o softmax padrão primeiro subtrai, de todos os elementos de cada linha de S , o valor máximo dentre esses elementos. Porém, carregar e conhecer todos os elementos de cada linha de S é incompatível com a técnica de processamento em blocos (tiling).

Para lidar com esse desafio, o FlashAttention utiliza uma técnica chamada online softmax. Essa técnica consiste em manter na memória rápida da GPU duas estatísticas parciais para cada linha da matriz S : o máximo parcial, que é o escore máximo visto até o momento, e o denominador parcial, que é a soma de todos os escores conhecidos até então. Fixado o input Q (query), a cada novo bloco dos inputs K (key) e V (value) processado, o FlashAttention procede da seguinte forma: (1) calcula o máximo local do bloco atual e então atualiza o máximo parcial; (2) calcula o denominador local do bloco atual utilizando o novo máximo parcial e então o soma ao denominador antigo, previamente reescalado com base na mudança dos máximos, para atualizar o denominador parcial; e (3) reescala o output acumulado até a iteração anterior por um determinado fator (baseado na mudança dos máximos e dos denominadores) e então o soma ao output local do bloco atual, obtendo assim o novo output acumulado. Ao final do loop, o valor do output na memória rápida da GPU é matematicamente idêntico ao que seria se o softmax fosse calculado de uma única vez (como no softmax padrão), mas sem nunca ter lido, da memória lenta, linhas inteiras da matriz S e sem nunca ter precisado de múltiplos passes sobre os dados.

QUESTÃO 3

ENUNCIADO:

Durante o passo de forward, o FlashAttention não salva as matrizes S (de escores de atenção, com tamanho $N \times N$) e P (de escores normalizados após a aplicação do softmax, com tamanho $N \times N$). O que ele salva em substituição a essas matrizes para preservar a capacidade de calcular os gradientes durante o passo de backward?

RESPOSTA:

Durante o passo de forward, a atenção padrão mantém salvas as matrizes S e P na memória principal da GPU, para uso posterior no passo de backward, porque essas matrizes são necessárias para calcular os gradientes com respeito às matrizes de entrada do componente de atenção.

O FlashAttention muda completamente essa abordagem por meio de uma estratégia baseada em recomputação. Durante o passo de forward, o FlashAttention nunca materializa as matrizes S e P em memória. Em vez disso, para o passo de backward, o FlashAttention mantém em memória apenas:

- Os inputs originais Q , K , V (matrizes de queries, keys e values), cada um de tamanho $N \times d$, onde N é o comprimento da sequência e d é a dimensão de cada cabeça de atenção, totalizando $O(Nd)$. Esses já estavam sendo salvos na atenção padrão de qualquer forma;
- As estatísticas do online softmax, especificamente o máximo e o denominador para cada uma das N linhas da matriz S de escores de atenção. Isso adiciona apenas $O(N)$ elementos adicionais ao estado salvo.

Durante o passo de backward, quando o FlashAttention precisa calcular os gradientes com respeito a determinados blocos dos inputs, ele simplesmente recalcula os blocos correspondentes das matrizes S e P sob demanda. Isso é feito carregando os blocos dos inputs Q , K , V da memória lenta para a memória rápida, e refazendo o passo de forward apenas para esses blocos. Uma vez que os blocos correspondentes de S e P são recomputados na memória rápida, eles são usados para o cálculo dos gradientes locais e imediatamente descartados, liberando a memória rápida para os próximos blocos. Embora isso aumente o número total de operações computacionais, essa estratégia remove completamente a leitura de matrizes de tamanho $O(N^2)$ da memória lenta, provando que é muito mais rápido repetir cálculos na memória rápida do que esperar pela movimentação de dados envolvendo a memória lenta.

QUESTÃO 4

ENUNCIADO:

A complexidade computacional do FlashAttention continua sendo quadrática em relação ao tamanho das sequências, assim como a da atenção padrão. Por que, então, o FlashAttention é muito mais rápido em tempo de execução?

RESPOSTA:

A razão para a redução do tempo de execução proporcionado pelo FlashAttention, mesmo com a mesma complexidade computacional $O(N^2)$, reside na diferença fundamental entre ser "memory-bound" (limitado por memória) e "compute-bound" (limitado por computação). A atenção padrão, ao ler e escrever na memória lenta as matrizes S (de escores de atenção) e P (de escores normalizados), ambas de tamanho $N \times N$, era "memory-bound". O tempo de execução era dominado pelo I/O, isto é, pelo o tempo de espera para ler e escrever na memória lenta, e não pela quantidade de cálculos realizados na memória rápida. As unidades de computação da GPU ficavam ociosas a maior parte do tempo, esperando os dados chegarem ou serem salvos.

O FlashAttention é um algoritmo "IO-aware". Ao usar tiling (processamento em blocos) e recomputação, ele elimina completamente as leituras e escritas de matrizes de tamanho $O(N^2)$ da memória lenta da GPU. Todas as operações intermediárias, com complexidade computacional de $O(N^2)$, acontecem dentro da memória rápida da GPU. Isso muda a natureza do gargalo: a operação deixa de ser "memory-bound" e passa a ser "compute-bound". Agora, o tempo de execução é ditado pela velocidade com que a GPU consegue executar as operações $O(N^2)$. Como as GPUs modernas têm um poder de computação massivo (altíssimo TFLOPs/s) que é muito superior à largura de banda da sua memória lenta, executar os cálculos $O(N^2)$ na memória rápida consome menos tempo do que ficar esperando pelo I/O de complexidade $O(N^2)$ da memória lenta. Essencialmente, o FlashAttention trocou um gargalo de memória por um gargalo de computação, permitindo que a GPU finalmente usasse seu potencial máximo.

QUESTÃO 5

ENUNCIADO:

O FlashAttention permite treinar não apenas modelos mais rápidos, mas também modelos melhores. Como a otimização de memória do FlashAttention leva a modelos com melhor eficácia (ex: menor perplexidade ou melhor acurácia)?

RESPOSTA:

A otimização de memória do FlashAttention é o que diretamente possibilita "modelos melhores". A complexidade de memória $O(N^2)$ da atenção padrão impunha um limite no comprimento máximo da sequência (N) que cabia na memória de uma GPU, geralmente limitado a 1k ou 2k de tokens. Qualquer sequência maior do que isso (por exemplo, 16k) exigiria dezenas de gigabytes de memória por cabeça de atenção em cada camada, o que era inviável.

O FlashAttention, ao eliminar a materialização (alocação) das matrizes S (de escores de atenção) e P (de escores normalizados), ambas de tamanho $N \times N$, reduz a necessidade de memória para $O(N)$ (complexidade linear). O limite de memória deixa de ser ditado por $O(N^2)$ e passa a ser ditado por $O(Nd)$ (para armazenar os inputs Q (query), K (key), V (value)), onde d é a dimensão de cada cabeça de atenção, e $O(N)$ (para as estatísticas do online softmax). Isso quebra a barreira do N^2 e permite, pela primeira vez, treinar modelos com comprimentos de sequência muito maiores (4k, 8k, 16k, e até 64k no caso do block-sparse).

A capacidade de treinar com um contexto maior é o que leva a modelos melhores. Um modelo com contexto 4k pode ingerir 4096 tokens de uma sequência para prever o próximo token, enquanto um modelo baseado na atenção padrão só podia ingerir 1024 tokens. Isso permite ao modelo aprender dependências de longo prazo, entender a estrutura de documentos inteiros, ou acompanhar narrativas complexas. Como mostrado nos resultados do paper do FlashAttention-1, o GPT-2 treinado com contexto 4k alcançou uma perplexidade menor, e em tarefas de classificação de documentos longos, o contexto maior levou a um ganho significativo de acurácia.